Response Code

Before response codes became part of HTTP Response Message, the client decided the status of the Request, whether the response is success or a failure, based on the body of the response.

Response Code is a three-digit number that helps the client to know what happened with your request in a standardized way.

Response codes fall into 5 different ranges.

1XX - Informational Response

2XX - Success Response

3XX - Redirection Response

4XX - Client Side Error

5XX - Server Side Error

Informational Response

These types of response codes are just sent as a request to let the client know about something. For example, a client wants to send a large file, but before that, it has to know whether the server is ready or not, so the client only sends the Request Headers, and then the server replies with an informational response.

Success Response

As the name suggests, the success response codes are used when the response is a successful response to let our client know the required action is successfully performed by the server. Let's see the most commonly used response codes.

`200 OK`

`Meaning:`
The request succeeded and the server is returning the requested data.

`Typical usage:`

- Successful **`GET`**

- Successful **PUT** (full update)
- Successful **POST** when returning a response body

Key point:
Use when you **`return a response body`** and everything worked as expected.

Real-world example:

```
// User clicks on "View Profile" button
GET /api/users/123

// Server responds with 200 and user data
{
  "id": 123,
  "name": "Ahmed",
  "email": "ahmed@example.com"
}
```

Jaise jab aap Facebook pe apni profile open karte hain, to Facebook ka server apka data bhejta hai with 200 OK status.

201 Created

Meaning:
A new resource was successfully created.

Typical usage:

- Successful **POST** that creates a new resource

Best practice:

- Include a **`Location`** header pointing to the new resource
- Optionally include the created object in the response body

Key point:
Use **`only when something new is created`**, not for updates.

Real-world example:

```
// User creates a new blog post
POST /api/posts
{
  "title": "My First Blog",
  "content": "Hello World!"
}

// Server responds with 201 Created
Response: 201 Created
Location: /api/posts/456
{
  "id": 456,
  "title": "My First Blog",
  "content": "Hello World!",
  "createdAt": "2025-02-02T10:30:00Z"
}
```

Jaise jab aap Instagram pe nai post share karte hain, to Instagram ek naya post create karta hai aur 201 return karta hai.

204 No Content

Meaning:
The request succeeded, but there is **nothing to return**.

Typical usage:

- Successful **DELETE**
- Successful **PUT/PATCH** when you don't want to return data
- Actions that don't require a response body

Important rules:

- **No response body allowed**
- No JSON, no message, no whitespace

Key point:
Use when success is confirmed but the client doesn't need data back.

Real-world example:

```
// User deletes a comment
DELETE /api/comments/789

// Server responds with 204 (no body)
Response: 204 No Content
(empty response body)
```

Jaise jab aap WhatsApp pe koi message delete karte hain, to server sirf confirm karta hai ke delete ho gaya, lekin koi data wapas nahi bhejta.

3XX Redirection Response

When a resource is moved to a new location then we use these response codes to redirect the client to the new URL.

The most commonly used responses in this range are:

301 Permanent Redirect

That means the resource is permanently moved to a new location. For example, you had an endpoint with the name of `users` and now you moved it to `persons` but for backward compatibility you don't want your clients who are hitting the `/users` endpoint to have an error, so whenever your client hits the `/users` endpoint you redirect your client to the newly created endpoint `/persons` with status code 301.

Real-world example:

```
// Old URL
GET /api/v1/users/123

// Server responds with 301
Response: 301 Moved Permanently
Location: /api/v2/persons/123
```

Jaise jab koi website permanently apna domain change kar deti hai (example.com se newexample.com), to purane link pe 301 redirect lagta hai.

302 Temporary Redirect

Iska mtlb hai ke resource temporarily ek new URL pe move ho gaya hai per client purane URL ko use kar sakta hai future request ke liye.

Real-world example:

```
// Website under maintenance
GET /api/products

// Server temporarily redirects to maintenance page
Response: 302 Temporary Redirect
Location: /maintenance.html
```

Jaise jab koi website maintenance ke liye temporarily band hoti hai aur users ko "We'll be back soon" page pe redirect kar deti hai.

304 Not Modified

Yeah caching mein use hota hai jab server batata hai last time jab tumne cache kiya tha response uske baad se resource wesa hi hai usme koi modification nahi hui.

Real-world example:

```
// Browser requests a logo image
GET /images/logo.png
If-Modified-Since: Mon, 01 Feb 2025 10:00:00 GMT

// Server checks and finds image hasn't changed
Response: 304 Not Modified
(browser uses cached version)
```

Jaise jab aap koi website dobara visit karte hain to images aur CSS files cache se load hoti hain kyunke woh change nahi hui. Server 304 bhej kar kehta hai "apne cache wali file use karo".

4XX Client Side Error

400 Bad Request

Jab client request ko sahi se nahi karta tab yeah response code use hota hai for example app body mein name as string expect kar rahe hain aur user ne array bhej diya to yeah ek bad request hui.

Real-world example:

```
// API expects email but user sends invalid format
POST /api/register
{
  "email": "not-an-email",  // Invalid email format
  "password": "12345"
}

// Server responds
Response: 400 Bad Request
{
  "error": "Invalid email format"
}
```

Jaise registration form mein email ki jagah kuch aur likhna, ya required field ko khali chhod dena. Server 400 return karega.

401 Unauthorized

Jab client apni authentication detail provide nahi karta ya phir karta hai to woh sahi nahi hoti to tab hum use 401 se respond karte hain.

For example server is expecting a JWT token, but the client never sent it, or sent it, but it's expired. In that case server responds with 401.

Real-world example:

```
// User tries to access protected route without login
GET /api/profile
// No Authorization header sent

// Server responds
Response: 401 Unauthorized
{
  "error": "Authentication required"
}
```

Jaise jab aap bina login kiye apne Twitter profile ko edit karne ki koshish karte hain, to 401 error aata hai kyunke aap logged in nahi hain.

403 Forbidden

Jab server ne apki request ko recognize kar liya as a legitimate user, but jo resource aap access karna chah rahe hain uski apko permission nahi hai, tab server 403 se respond karta hai.

Real-world example:

```
// Regular user tries to delete another user
DELETE /api/users/999
Authorization: Bearer valid_token_for_user_123

// Server responds
Response: 403 Forbidden
{
  "error": "You don't have permission to delete other users"
}
```

Jaise jab aap Google Drive mein kisi aur ki file ko delete karne ki koshish karte hain. Aap logged in hain (401 nahi hai) lekin permission nahi hai (403 hai).

404 Not Found

Jab client aise resource request karta hai jo exist nahi karta server pe ya delete ho chuka in that case server client ko 404 Not Found ke response code and message se respond karta hai.

Real-world example:

```
// User tries to access non-existent product
GET /api/products/99999

// Server responds
Response: 404 Not Found
{
  "error": "Product not found"
}
```

Jaise jab aap kisi purane YouTube video ka link open karte hain jo ab delete ho chuka hai, to "Video not found" dikhai deta hai. Yeah 404 error hai.

405 Method Not Allowed

Jab ek resource jo sirf ek GET ya POST method hi support karta ho aap use PUT ya PATCH se request kar dein to tab server 405 se response karta hai ke jo method se aap request kar rahe woh yeah particular resource support nahi karta.

Real-world example:

```
// Trying to DELETE a resource that only allows GET
DELETE /api/public-info

// Server responds
Response: 405 Method Not Allowed
Allow: GET
{
  "error": "DELETE method not allowed on this resource"
}
```

Jaise ek read-only API endpoint hai jo sirf data dikhata hai. Agar aap us endpoint se DELETE request bhejte hain to 405 error aata hai kyunke woh endpoint sirf GET support karta hai.

409 Conflict

409 Conflict tab use hota hai jab client ki request ya us ke bheje gaye data ka server ki current state ya existing data ke sath conflict ho.

Misal ke taur par, agar user kisi aise email se register karne ki koshish kare jo pehle hi system mein mojood ho, to request valid hoti hai lekin complete nahi ki ja sakti kyun ke woh existing data se conflict karti hai, is liye **409 Conflict** return kiya jata hai.

Real-world example:

```
// User tries to register with existing email
POST /api/register
{
  "email": "ahmed@example.com",  // Email already exists
  "password": "password123"
}

// Server responds
Response: 409 Conflict
{
  "error": "Email already registered"
}
```

Ya jaise GitHub pe jab aap ek repository create karte hain aur wahi naam pehle se mojood hai, to conflict hota hai.

429 Too Many Requests

Yeah, rate limiting mein use hota hai. Let's say ek user 20 minutes mein sirf 3 login attempts hi kar sakta hai isse zyada allowed nahi hai.

Ab agar client authentication ke liye 20 minutes mein 4 request kar deta hai to 4th request pe server error dega with response code 429 Too Many Requests.

Real-world example:

```
// User tries 4th login in 20 minutes
POST /api/login  // 4th attempt
{
  "email": "user@example.com",
  "password": "wrong"
}

// Server responds
Response: 429 Too Many Requests
Retry-After: 1200  // Wait 1200 seconds (20 minutes)
{
  "error": "Too many login attempts. Try again in 20 minutes"
}
```

Jaise Twitter ya Instagram pe jab aap bahut jaldi jaldi posts share karte hain to "You're doing this too fast" message aata hai. Yeah 429 error hai jo spam rokne ke liye use hota hai.

5XX Server Side Error

500 Internal Server Error

Jab server ki side koi unexpected error ata hai to tab yeah send kiya jata hai takey client ko pata bhi chal jaye aur request hang bhi nah ho.

Real-world example:

```
// User requests data but server has a bug
GET /api/orders

// Server responds
Response: 500 Internal Server Error
{
  "error": "Something went wrong on our end"
}

// In server logs:
// TypeError: Cannot read property 'price' of undefined
```

Jaise jab aap koi website khol rahe hain aur "Something went wrong" ya "Oops! We're having technical difficulties" dikhai deta hai. Yeah server-side bug ya database connection issue ki wajah se hota hai.

501 Not Implemented

Jab server koi ek feature ya HTTP header ya method support nahi karta but it plans to add it soon tab server 501 code ke sath respond karta hai.

Real-world example:

```
// API doesn't support PATCH yet
PATCH /api/users/123
{
  "name": "New Name"
}

// Server responds
Response: 501 Not Implemented
{
  "error": "PATCH method not implemented yet. Use PUT instead"
}
```

Jaise ek naya API hai jo abhi sirf basic features support karta hai. Agar aap advanced feature use karne ki koshish karte hain jo abhi develop nahi hua to 501 milega.

502 Bad Gateway

502 Bad Gateway error tab use hota hai jab ek server (jo gateway ya proxy ka kaam kar raha ho) ko peeche wale (upstream) server se invalid response milta hai.

Real-world example:

```
// Your app server tries to connect to payment gateway
POST /api/checkout

// Payment gateway returns corrupted response
// Your server responds to client
Response: 502 Bad Gateway
{
  "error": "Payment service returned invalid response"
}
```

Jaise jab aap online payment kar rahe hain aur payment gateway (JazzCash, EasyPaisa) properly respond nahi kar raha ya crash ho gaya hai. Tumhari website 502 error show karegi.

503 Service Not Available

Jab server high traffic ki wajah se ek request ko handle nahi kar sakta ya phir during maintenance jab request ko entertain nahi kar sakte to tab Service Not Available ko use kiya jata hai.

Real-world example:

```
// Server is overloaded or under maintenance
GET /api/products

// Server responds
Response: 503 Service Unavailable
Retry-After: 3600  // Try again after 1 hour
{
  "error": "Server is temporarily unavailable. Please try again later"
}
```

Jaise Black Friday sale ke time Amazon ya Daraz ki website slow ho jati hai ya crash kar jati hai kyunke bahut zyada log ek sath shopping kar rahe hote hain. Tab 503 error aata hai.

504 Gateway Timeout

504 Gateway Timeout error tab aata hai jab ek server (jo gateway ya proxy ka kaam kar raha ho) ko apne piche wale (upstream) server se **`waqt par response nahi milta`**.

Dono 502 aur 504 mein farq ye hai ke 502 mein response "invalid" hota hai, jabke 504 mein response "aaya hi nahi" (timeout ho gaya).

Real-world example:

```
// Your server waits for database response
GET /api/reports/generate

// Database takes too long (30+ seconds)
// Your server gives up and responds
Response: 504 Gateway Timeout
{
  "error": "Request timeout - database took too long to respond"
}
```

Jaise jab aap koi bahut bari file upload kar rahe hain aur internet slow hai. Server ek limit tak wait karta hai (e.g., 30 seconds), agar tab tak response nahi aya to 504 timeout error de deta hai.

502 vs 504 ka farq:

- **502**: Server ne kuch galat response diya (garbage data)
- **504**: Server ne koi response hi nahi diya (bahut wait karne ke baad bhi)